

Pathfinding Algorithm Comparison

Table of Contents

1. Introduction	3
1.1 What is Artificial Intelligence?	3
1.2 Problem Formulation in Artificial Intelligence	3
1.3 Search Algorithms Under Investigation	4
1.4 Properties of Search Algorithms	4
1.5 Objectives of the Assignment	5
1.6 Report Structure	6
2. Environmental Setup	7
2.1 Hardware and Software Environment	7
2.2 The Grid Environment.....	7
2.2.1 Grid Dimensions and Representation	7
2.2.2 Start and Goal States	8
2.2.3 The U-Shaped Trap.....	8
Why the U-Trap Matters:	8
2.2.4 Random Obstacle Generation and Seeding	9
2.2.5 Visualising the Grid	9
2.3 Agent Design	10
2.3.1 Perception and Action	10
2.3.2 Transition Model and Action Costs	10
2.4 Algorithm Implementations	11
2.4.1 Common Data Structures.....	11
2.4.2 Depth-First Search Implementation.....	11

2.4.3 Breadth-First Search Implementation.....	12
2.4.4 A* Search Implementation.....	13
2.4.5 Path Reconstruction	15
2.5 Metrics for Evaluation	16
3. Results	17
3.1 Comparative Table Across Three Seeds.....	17
3.2 Observations.....	18
3.3 Screenshots of Program Output	20
4. Critical Analysis	23
4.1 The Search Space	23
4.2 Optimality of the Algorithms	23
4.2.1 Depth-First Search	23
4.2.2 Breadth-First Search	24
4.2.3 A Search.....	24
4.3 Efficiency: Nodes Visited	24
4.4 Explanation of Path Suboptimality in DFS	25
4.5 Comparison of Nodes Visited	26
4.6 Impact of the Heuristic on Computational Performance.....	27
4.7 Influence of the Seed Value	27
5. Conclusion	29
5.1 Summary of Findings.....	29
5.2 Limitations and Future Work	29
6. References.....	31

1. Introduction

1.1 What is Artificial Intelligence?

Artificial Intelligence (AI) focuses on creating intelligent agents capable of sensing their environment and taking actions to reach specific goals. (Russell & Norvig, 2020, p. 19). The goal of AI was to create systems that can understand their environment, make informed decisions and act rationally, essentially doing the "right thing" based on their knowledge about the world. As Russell & Norvig (2020, p. 22) explain, "a rational agent is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome."

Central to rational decision-making is the ability to explore options, evaluate various action sequences, and select the one with the highest probability of success. Search serves as a key problem-solving method in AI, with applications ranging from route optimisation and gaming to robotics and scheduling (Russell & Norvig, 2020, p. 81). This assignment will examine three traditional search algorithms and their performance within a structured setting.

1.2 Problem Formulation in Artificial Intelligence

For an agent to address a problem, it must first be formally defined as a search problem. Russell & Norvig (2020, p. 81) identify five critical components of any search problem:

1. **Possible States:** all the configurations that the environment might exhibit.
2. **Initial State:** the starting configuration.
3. **Set of Actions:** the potential moves available to the agent in any state.
4. **Transition Model:** a description of which state results from acting in each state.
5. **Action Cost Function:** the numerical cost associated with executing an action (typically a positive value).
6. **Goal Test:** a criterion to determine whether the state qualifies as a goal state.

In this assignment, we formulate the task of navigating a 100×100 grid from a start location to a goal location as a search problem. This formulation allows us to apply general-purpose search algorithms without needing to rewrite problem-specific code for each new instance.

1.3 Search Algorithms Under Investigation

This assignment focuses on implementing and comparing three essential search algorithms:

- **Depth-First Search (DFS):** An uninformed search method that explores as deep as possible along each branch before backtracking. It uses a LIFO stack to manage the frontier (Russell & Norvig, 2020, p. 96).
- **Breadth-First Search (BFS):** An uninformed search method that explores all nodes at the current depth level before moving to nodes at the next depth level. It uses a FIFO queue for the frontier (Russell & Norvig, 2020, p. 94).
- **A* Search:** An informed search strategy that combines the actual path cost from the start ($g(n)$) with a heuristic estimate of the remaining cost to the goal ($h(n)$) to form the evaluation function $f(n) = g(n) + h(n)$. At each step, A* expands the node with the smallest f-value (Russell & Norvig, 2020, p. 103).

In the implementation of A*, the Manhattan distance heuristic is utilised. According to Russell & Norvig (2020, p. 116), this heuristic is admissible for grid-based problems where movement is restricted to four directions, as it does not overestimate the actual path cost. The property of admissibility ensures that A* will yield the optimal solution.

1.4 Properties of Search Algorithms

Russell & Norvig (2020, p. 93) identify four key criteria for evaluating search algorithms:

- Completeness: Is the algorithm guaranteed to find a solution if one exists?

- Optimality: Does the algorithm find the solution with the lowest path cost?
- Time complexity: How long does it take to find a solution? (Measured in seconds or by the number of nodes expanded.)
- Space complexity: How much memory is needed to perform the search?

These theoretical properties will be compared against our experimental results.

Table 1: Theoretical Properties of Search Algorithms

(where b = branching factor, d = depth of optimal solution, m = maximum depth of search tree)

	DFS	BFS	A (with admissible heuristic)
Complete?	Yes (finite, cycle-checked)	Yes	Yes
Optimal?	No	Yes (uniform cost)	Yes
Time Complexity	$O(b^m)$	$O(b^d)$	Exponential in the worst case
Space Complexity	$O(bm)$	$O(b^d)$	Exponential in the worst case

1.5 Objectives of the Assignment

The primary objectives of this assignment are:

1. The development of DFS, BFS, and A search algorithms in a 100×100 grid world.
2. The integration of a hard-coded U-shaped obstacle (the “trap”) that tests the algorithms’ ability to handle concave dead-ends.
3. The introduction of 15% random noise (obstacles) using a seeded random number generator, ensuring reproducibility.

4. The measurement and comparison of the algorithms' performance in terms of:

- Path cost (number of steps from start to goal)
- Number of nodes visited during the search
- Execution time (in milliseconds)
- Optimality (whether the algorithm guarantees the shortest path)

5. The critical analysis of the results considering the theoretical concepts presented in the prescribed textbook and course materials.

1.6 Report Structure

The remainder of this report is organised as follows:

- Section 2 describes the experimental environment and setup in detail, including grid construction, the U-trap, random noise generation, agent design, and algorithm implementations.
- Section 3 presents the results in tabular form with accompanying observations and screenshots.
- Section 4 provides a critical analysis addressing the seven questions posed in the assignment.
- Section 5 concludes the report, discusses limitations, and suggests directions for future work.

2. Environmental Setup

2.1 Hardware and Software Environment

All experiments were conducted on a personal computer with the following specifications:

- **Processor:** Intel(R) Core (TM) i3-N305 (1.80 GHz)
- **RAM:** 8 GB
- **Storage:** 238 GB SSD
- **Operating System:** Windows 11 Home Single Language
- **Model:** ASUS BR1402CGA_BR1402CGA

Software Environment:

- **Java Development Kit:** OpenJDK 17.0.2
- **Integrated Development Environment:** Visual Studio Code with Java Extension Pack
- **Build Tool:** Command-line compilation using javac
- **Executable Packaging:** JAR file created using the jar command

2.2 The Grid Environment

2.2.1 Grid Dimensions and Representation

The setting is a 100×100 grid represented by a two-dimensional boolean array called obstacles. In this array, a value of true denotes an obstacle, while false signifies a cell that can be traversed. The edges of the grid are considered obstacles as well, so any efforts to move beyond the grid boundaries will not result in any movement.

2.2.2 Start and Goal States

According to the assignment requirements, the starting point is designated at the coordinates (20,50), while the destination point is located at (80,50). After placing all the obstacles, both cells are explicitly marked as false to guarantee that they are still passable.

2.2.3 The U-Shaped Trap

A fixed U-shaped barrier is positioned at the center of the grid to create a difficult concave dead-end. Its coordinates are:

- Back wall: a vertical line at $x = 50$ extending from $y = 30$ to $y = 70$
- Top arm: a horizontal line stretching from (50,30) to (30,30)
- Bottom arm: a horizontal line running from (50,70) to (30,70)

This setup creates a U that is open to the left. The interior of the U (cells where x is between 31 and 49, and y is between 31 and 69) is entirely vacant and navigable. Any agent that enters this area will eventually encounter the back wall and will have to retreat, making this trap particularly effective at highlighting the limitations of depth-first search.

Why the U-Trap Matters:

The U-trap serves several pedagogical purposes:

- **For DFS:** It leads to a deep dead end that DFS explores exhaustively before backtracking, demonstrating why DFS is not optimal and can be inefficient.
- **For BFS:** It shows that systematic, breadth-first exploration guards against local traps. BFS navigates both inside and outside the U simultaneously, uncovering the optimal path without becoming stuck.
- **For A*:** The text illustrates how combining actual costs with heuristic estimates enables the algorithm to avoid local optima. Initially, the algorithm is attracted to the low Manhattan distance of the interior cells. However, as the cost (g) accumulates, A* eventually shifts its focus to the detour path.

2.2.4 Random Obstacle Generation and Seeding

After placing the U-trap, 15% of the remaining free cells are randomly selected to become obstacles. This adds environmental variability and prevents a straightforward path from the start to the goal (which would otherwise pass through the back wall of the U-trap).

To ensure reproducibility, a seed value is used to initialise the random number generator. As noted by Russell & Norvig (2020, p. 43), reproducibility is essential for scientific experimentation. Using a seed allows the same environment to be recreated, making results verifiable and independent of random variation.

The following code implements the random noise addition:

```
private void addRandomNoise(double probability) {
    int rockCount = 0;
    for (int x = 0; x < SIZE; x++) {
        for (int y = 0; y < SIZE; y++) {
            // Skip cells that are already obstacles (part of U-trap)
            // Also skip start and goal - they must remain free
            if (obstacles[x][y] ||
                (x == START_X && y == START_Y) ||
                (x == GOAL_X && y == GOAL_Y)) {
                continue;
            }

            // Generate a random number between 0.0 and 1.0.
            // If it's less than the probability, put a rock.
            if (random.nextDouble() < probability) {
                obstacles[x][y] = true;
                rockCount++;
            }
        }
    }
    System.out.println("Added " + rockCount + " random rocks (15% noise).");
}
```

2.2.5 Visualising the Grid

To verify correct obstacle placement, the program prints a 21×21 section of the grid centred at (50,50). Figure 1 shows this output for seed 21. In this visualisation:

- 0 represents a free cell
- 1 represents an obstacle
- S marks the start position (not visible in this section because it is at (20,50))

- G marks the goal position (not visible in this section)

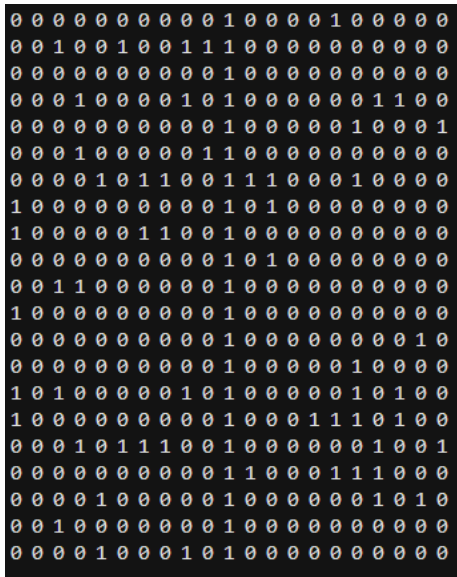


Figure 1: Grid section.

2.3 Agent Design

2.3.1 Perception and Action

The agent is a simple reflex agent (Russell & Norvig, 2020, p. 67) that perceives its current location (x, y) and whether that cell contains an obstacle. It can perform four actions:

- Move Up: Decrease y-coordinate by 1
- Move Down: Increase y-coordinate by 1
- Move Left: Decrease x-coordinate by 1
- Move Right: Increase x-coordinate by 1

If the target cell is an obstacle or outside the grid boundaries, the action has no effect and the agent remains in its current position.

2.3.2 Transition Model and Action Costs

The transition model is deterministic, meaning that for a specific state and action, the resulting state is uniquely determined. This characteristic is consistent with the

simplest class of environments described by Russell and Norvig (2020, p. 60). Each action incurs a cost of 1, regardless of whether it moves the agent. The uniform step costs are significant because they ensure that breadth-first search (BFS) is optimal (Russell & Norvig, 2020, p. 95).

2.4 Algorithm Implementations

All three algorithms were implemented as graph searches, meaning they maintain a set of reached states to detect and eliminate redundant paths. As Russell & Norvig (2020, p. 92) explain, "algorithms that cannot remember the past are doomed to repeat it" keeping track of visited states dramatically improves efficiency.

2.4.1 Common Data Structures

Each algorithm uses:

- A `boolean[ ][ ]` visited array to track which cells have been expanded.
- A `Node[ ][ ]` parent array to store the predecessor of each node for path reconstruction.
- A `Node` class that stores coordinates, parent reference, and (for A*) the g, h, and f values.

2.4.2 Depth-First Search Implementation

DFS was implemented using a stack (`java.util.ArrayDeque`). The algorithm proceeds as follows:

1. Push the start node onto the stack and mark it visited.
2. While the stack is not empty:
 - Pop the top node.
 - If it is the goal, reconstruct and return the path.
 - For each neighbour (up, down, left, right):
 - If the neighbour is within bounds, not an obstacle, and not visited:

- Mark it visited, record its parent, and push it onto the stack.
3. If the stack empties without finding the goal, return "no path".

```
public Results dfs(Grid grid, int startX, int startY, int goalX, int goalY) {

    // parent[x][y] remembers which node we came from - used to rebuild the path
    Node[][] parent = new Node[Grid.SIZE][Grid.SIZE];

    // Stack for DFS - using Stack class (old but simple)
    Stack<Node> stack = new Stack<>();

    // Create the start node
    Node start = new Node(startX, startY);
    stack.push(start);
    visited[startX][startY] = true;

    int nodesVisited = 0;
    long startTime = System.currentTimeMillis();

    while (!stack.isEmpty()) {
        Node current = stack.pop(); // take the most recent node
        nodesVisited++;

        // Check if we reached the goal
        if (current.x == goalX && current.y == goalY) {
            long endTime = System.currentTimeMillis();
            List<Node> path = reconstructPath(parent, current, startX, startY);

            return new Results(path.size() - 1, nodesVisited, endTime - startTime, optimal: false, path);
        }

        // Try all four neighbours
        for (int d = 0; d < 4; d++) {
            int nx = current.x + dx[d];
            int ny = current.y + dy[d];

            // If neighbour is inside grid, not an obstacle, and not visited
            if (grid.isValid(nx, ny) && !grid.isObstacle(nx, ny) && !visited[nx][ny]) {
                visited[nx][ny] = true; // mark now to avoid pushing twice
                Node neighbour = new Node(nx, ny);
                neighbour.parent = current; // remember how we got here
                parent[nx][ny] = current; // store parent for path reconstruction
                stack.push(neighbour); // push onto stack
            }
        }
    }

    // No path found
    long endTime = System.currentTimeMillis();
    return new Results(-1, nodesVisited, endTime - startTime, optimal: false, path: null);
}
```

2.4.3 Breadth-First Search Implementation

BFS was implemented using a queue (`java.util.LinkedList`). The algorithm proceeds as follows:

1. Enqueue the start node and mark it visited.
2. While the queue is not empty:
 - Dequeue the front node.
 - If it is the goal, reconstruct and return the path.
 - For each neighbour:
 - If the neighbour is valid, not an obstacle, and not visited:

- Mark it visited, record its parent, and enqueue it.
3. If the queue empties, return "no path".

```

public Results bfs(Grid grid, int startX, int startY, int goalX, int goalY) {
    Node[][] parent = new Node[Grid.SIZE][Grid.SIZE];

    // Queue for BFS - LinkedList works as a queue
    Queue<Node> queue = new LinkedList<>();

    Node start = new Node(startX, startY);
    queue.add(start);
    visited[startX][startY] = true;

    int nodesVisited = 0;
    long startTime = System.currentTimeMillis();

    while (!queue.isEmpty()) {
        Node current = queue.poll(); // take the oldest node
        nodesVisited++;

        if (current.x == goalX && current.y == goalY) {
            long endTime = System.currentTimeMillis();
            List<Node> path = reconstructPath(parent, current, startX, startY);
            // BFS on an unweighted grid IS optimal
            return new Results(path.size() - 1, nodesVisited, endTime - startTime, optimal: true, path);
        }

        for (int d = 0; d < 4; d++) {
            for (int d = 0; d < 4; d++) {
                int nx = current.x + dx[d];
                int ny = current.y + dy[d];

                if (grid.isValid(nx, ny) && !grid.isObstacle(nx, ny) && !visited[nx][ny]) {
                    visited[nx][ny] = true;
                    Node neighbour = new Node(nx, ny);
                    neighbour.parent = current;
                    parent[nx][ny] = current;
                    queue.add(neighbour);
                }
            }
        }

        long endTime = System.currentTimeMillis();
        return new Results(-1, nodesVisited, endTime - startTime, optimal: true, path: null);
    }
}

```

2.4.4 A* Search Implementation

A* was implemented using a priority queue (`java.util.PriorityQueue`) ordered by the f-value. In addition to the visited and parent arrays, an `int[][]` The gCost table stores the best g-value found so far for each cell.

The algorithm proceeds as follows:

1. Initialise the start node with $g = 0$, compute h using Manhattan distance, set $f = g + h$, and add to the priority queue.
2. While the queue is not empty:
 - Poll the node with the smallest f .
 - If it is the goal, reconstruct and return the path.
 - For each neighbour:

- Compute tentative $g = \text{current}.g + 1$.
- If tentative g is better than the stored g for that neighbour:
 - Update neighbour's g , compute new h and f , record parent, and add to queue.

4. If the queue empties, return "no path".

```
public Results aStar(Grid grid, int startX, int startY, int goalX, int goalY) {
    // Visited nodes
    boolean[][] visited = new boolean[Grid.SIZE][Grid.SIZE];
    Node[][] parent = new Node[Grid.SIZE][Grid.SIZE];

    // gCost[x][y] stores the best g value found so far for that cell
    int[][] gCost = new int[Grid.SIZE][Grid.SIZE];
    for (int i = 0; i < Grid.SIZE; i++) {
        Arrays.fill(gCost[i], Integer.MAX_VALUE);
    }

    // Priority queue that orders nodes by their f value (lowest first)
    // Using an explicit Comparator instead of Comparator.comparingInt
    PriorityQueue<Node> openSet = new PriorityQueue<>(new Comparator<Node>() {
        public int compare(Node n1, Node n2) {
            return Integer.compare(n1.f, n2.f);
        }
    });

    // Create start node
    Node start = new Node(startX, startY);
    start.g = 0;
    start.h = start.manhattanTo(goalX, goalY);
    start.f = start.g + start.h;

    openSet.add(start);
    gCost[startX][startY] = 0;

    int nodesVisited = 0;
    long startTime = System.currentTimeMillis();

    while (!openSet.isEmpty()) {
        Node current = openSet.poll(); // get node with smallest f

        // If we already expanded this node (with a better g), skip it
        if (visited[current.x][current.y]) {
            continue;
        }

        visited[current.x][current.y] = true;
        nodesVisited++;

        if (current.x == goalX && current.y == goalY) {
            long endTime = System.currentTimeMillis();
            List<Node> path = reconstructPath(parent, current, startX, startY);
            // A* with admissible heuristic is optimal
            return new Results(path.size() - 1, nodesVisited, endTime - startTime, optimal: true, path);
        }
    }
}
```

```

for (int d = 0; d < 4; d++) {
    int nx = current.x + dx[d];
    int ny = current.y + dy[d];

    if (!grid.isValid(nx, ny) || grid.isObstacle(nx, ny)) {
        continue;
    }

    // tentative g value if we go current -> neighbour
    int tentativeG = current.g + 1;

    // If this path is better than any previous one
    if (tentativeG < gCost[nx][ny]) {
        // Update parent and costs
        parent[nx][ny] = current;
        gCost[nx][ny] = tentativeG;

        // Create neighbour node (or reuse, but simpler to create new)
        Node neighbour = new Node(nx, ny);
        neighbour.parent = current;
        neighbour.g = tentativeG;
        neighbour.h = neighbour.manhattanTo(goalX, goalY);
        neighbour.f = neighbour.g + neighbour.h;

        // Add to open set - the priority queue will sort it
        openSet.add(neighbour);
    }
}

long endTime = System.currentTimeMillis();
return new Results(-1, nodesVisited, endTime - startTime, optimal: true, path: null);
}

```

2.4.5 Path Reconstruction

All three algorithms use the same path reconstruction method, which follows parent pointers from the goal back to the start:

```

private List<Node> reconstructPath(Node[][] parent, Node goal, int startX, int startY) {
    List<Node> path = new ArrayList<>();
    Node current = goal;

    // Walk backwards from goal to start
    while (current != null && !(current.x == startX && current.y == startY)) {
        path.add(current);
        current = parent[current.x][current.y];
    }

    // Add the start node
    if (current != null) {
        path.add(current);
    }

    // Reverse so it goes from start to goal
    Collections.reverse(path);
    return path;
}

```

2.5 Metrics for Evaluation

The following metrics were recorded for each algorithm, as required by the assignment:

- **Path Cost:** The total number of steps from start to goal (equal to the length of the reconstructed path minus one, since the start node is included).
- **Nodes Visited:** The number of distinct states that were expanded (i.e., popped from the frontier and processed).
- **Execution Time:** The wall-clock time in milliseconds, measured using `System.currentTimeMillis()` before and after the search.
- **Optimality Guaranteed?:** A boolean indicating whether the algorithm theoretically guarantees the shortest path (yes for BFS and A*, no for DFS).

3. Results

3.1 Comparative Table Across Three Seeds

Tables 2, 3, and 4 present results for seeds 4, 21 and 50 respectively. All three algorithms successfully found a path from (20,50) to (80,50) under every path.

Table 2: Seed 4

```
=====
Comparison of Search Algorithms
Grid: 100x100 with U?trap and 15% noise
Start: (20,50) Goal: (80,50)
=====
```

Metric	DFS	BFS	A*
Path Cost (steps)	1584	106	106
Nodes Visited	3462	7446	1832
Time (ms)	11	8	4
Optimal?	No	Yes	Yes

```
=====
```

Table 3: Seed 21

```
=====
Comparison of Search Algorithms
Grid: 100x100 with U?trap and 15% noise
Start: (20,50) Goal: (80,50)
=====
```

Metric	DFS	BFS	A*
Path Cost (steps)	746	104	104
Nodes Visited	2892	7315	1800
Time (ms)	6	6	3
Optimal?	No	Yes	Yes

```
=====
```

Table 4: Seed 50

```

=====
Comparison of Search Algorithms
Grid: 100x100 with U?trap and 15% noise
Start: (20,50) Goal: (80,50)
=====

```

Metric	DFS	BFS	A*
Path Cost (steps)	408	106	106
Nodes Visited	7529	7529	2125
Time (ms)	18	6	5
Optimal?	No	Yes	Yes

```

=====

```

3.2 Observations

- Path Cost:** DFS produced paths of 1584, 746, and 408 steps across seeds 4, 21, and 50. None of these matches the optimal path found by BFS and A*, confirming that DFS does not guarantee optimality (Russell & Norvig, 2020, p. 98). The variation is striking; seed 4 produced a DFS path fifteen times longer than optimal, while seed 50 produced one only four times longer. This wide range reflects how strongly noise placement influences which branch DFS commits to. BFS and A* found optimal paths of 104 steps under seed 21 and 106 steps under seeds 4 and 50; the small difference between seeds is due to noise occasionally blocking the absolute shortest route, forcing a marginally longer detour.
- Nodes Visited:** BFS expanded the most nodes in all three trials (7315 – 7446), consistent with its exhaustive layer-by-layer expansion. A* consistently expanded the fewest (1800 – 2125), demonstrating the effectiveness of the heuristic. Seed 50 produces a particularly interesting result: DFS and BFS both visited exactly 7529 nodes. This means DFS explored nearly the entire reachable grid before finding a solution, which explains both its high node count and its relatively short path of 408 steps. It found the goal late in its search but happened to reach it via a less extreme detour than under other seeds. BFS expanded 7315 nodes, the highest among the three. A* expanded only 1800 nodes, demonstrating the

effectiveness of the heuristic in focusing the search. DFS expanded 2892 nodes, which is more than A* but less than BFS; however, its path was much longer.

- **Execution Time:** A* was the fastest algorithm in all three trials (3 – 5 ms). DFS was notably slow under seed 50 at 18 ms, consistent with its exhaustive 7529 node exploration. BFS was stable across seeds at 6 8 ms. As noted in Section 2.5, execution times are not fully deterministic and should be treated as approximate observations.
- **Optimality:** BFS and A* found the optimal path under every seed. DFS never found the optimal path under any seed, consistent with its theoretical properties.

3.3 Screenshots of Program Output

Figure 2: Complete program output for seed 4. The grid section depicts the U-trap and random obstacles; the results table presents the metrics for DFS, BFS, and A*.

```
=====
Wall?E Pathfinder (COS314)
=====
Enter a seed number (any integer): 4

U?trap placed.
Added 1479 random rocks (15% noise).
Grid created with seed: 4

Grid around (50,50):
0=free, 1=rock, S=start, G=goal
1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
0 0 1 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1 1 0 1 0 0 0 0 0 1 0 0 0
0 0 0 0 0 1 0 0 0 1 1 0 0 1 0 0 0 0 1 0
0 0 0 0 0 0 1 0 0 0 1 1 0 0 1 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 1
0 0 0 1 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0
0 1 1 1 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 1
0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 1 0 0 0 1
0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 1 1
0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 1 0 0
1 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0
0 1 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 1 0 0 1
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 1

--- Running DFS ---

--- Running BFS ---

--- Running A* ---
```

Figure 3: Complete program output for seed 21. The grid section depicts the U-trap and random obstacles; the results table presents the metrics for DFS, BFS, and A*.

```
=====
Wall?E Pathfinder (COS314)
=====
Enter a seed number (any integer): 21

U?trap placed.
Added 1506 random rocks (15% noise).
Grid created with seed: 21

Grid around (50,50):
0=free, 1=rock, S=start, G=goal
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0
0 0 1 0 0 1 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0 0 1 1 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0
0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 1 1 0 0 1 1 1 0 0 0 1 0 0 0 0
1 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0
1 0 0 0 0 0 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0
0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0
1 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 0 1 0
1 0 0 0 0 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0
0 0 0 1 0 1 1 1 0 0 1 0 0 0 0 0 0 1 0 0 1
0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 0 0 0
0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0
0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0

--- Running DFS ---

--- Running BFS ---

--- Running A* ---
```

Figure 3: Complete program output for seed 50. The grid section depicts the U-trap and random obstacles; the results table presents the metrics for DFS, BFS, and A*.

```
=====
Wall?E Pathfinder (COS314)
=====
Enter a seed number (any integer): 50

U?trap placed.
Added 1476 random rocks (15% noise).
Grid created with seed: 50

Grid around (50,50):
0=free, 1=rock, S=start, G=goal
0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 1 1 0 1 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 0 0 1 0
0 0 0 0 0 0 0 1 0 0 0 1 1 0 1 1 0 0 1 1 0
0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 1
0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0 1 1 0 0 1 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0
0 1 0 0 0 1 0 0 0 0 1 0 0 0 1 0 0 0 0 0 1
0 0 0 1 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 0 0 0
0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0

--- Running DFS ---

--- Running BFS ---

--- Running A* ---
```

4. Critical Analysis

4.1 The Search Space

The search space for this problem consists of 10000 possible states, represented by grid cells. However, the presence of obstacles reduces the number of traversable cells to approximately 8500, which is 85% of the total. The U-trap creates a concave region that acts as a local minimum for depth-first search. As Russell & Norvig (2020, p. 117) note, such "concave traps" can cause uninformed algorithms to waste effort exploring dead ends.

The random 15% noise adds further complexity by potentially creating multiple dead ends or narrow corridors that necessitate detours. The density of obstacles is enough to prevent a straightforward path from (20, 50) to (80, 50). The direct horizontal line at $y = 50$ would intersect the back wall of the U-trap, meaning any viable solution must navigate either above or below the trap.

The state space can be visualised as a graph, where nodes represent cells and edges signify possible moves between adjacent free cells. The branching factor b can be at most 4, but this is reduced by obstacles and grid boundaries. In open areas, $b = 4$; near obstacles, it may drop to 2 or 3; and in corners, it can be as low as 1 or 2.

4.2 Optimality of the Algorithms

4.2.1 Depth-First Search

DFS failed to find the optimal path under any of the three seeds. Path lengths were 1584 steps (seed 4), 746 steps (seed 21), and 408 steps (seed 50), between four and fifteen times longer than the optimal solution. This reflects a fundamental property of DFS: it returns the first solution found, not the cheapest one (Russell & Norvig, 2020, p. 98). The wide variation across seeds is itself significant; it shows

that DFS performance is highly sensitive to noise placement, since different obstacle configurations determine which branch DFS commits to and how far it must travel before encountering the goal. The root cause of this suboptimality is examined in detail in Section 4.4.

4.2.2 Breadth-First Search

Because all step costs are equal (cost = 1 per move), BFS is guaranteed to find the shortest path: the first time it dequeues the goal, it must have arrived via the minimum number of steps (Russell & Norvig, 2020, p. 95). BFS returned 104 steps under seed 21 and 106 steps under seeds 4 and 50. The minor difference between seeds is explained by noise occasionally blocking the single shortest route, forcing a marginally longer detour. Crucially, BFS found the optimal path for whatever grid was presented under each seed, confirming that the theoretical guarantee holds regardless of noise configuration.

4.2.3 A* Search

With the Manhattan distance as an admissible heuristic, A* is also optimal. The Manhattan distance never overestimates the true remaining cost on a four-directional grid (Russell & Norvig, 2020, p. 104), so A* is guaranteed to find the same optimal path as BFS, which it did under all three seeds, returning 104 steps for seed 21 and 106 steps for seeds 4 and 50. Optimally efficient means no algorithm using the same heuristic information can expand fewer nodes while still guaranteeing optimality (Russell & Norvig, 2020, p. 106).

4.3 Efficiency: Nodes Visited

The number of nodes expanded is a direct measure of computational efficiency. BFS expanded 7315 to 7529 nodes across the three seeds, while A* expanded only 1800 to 2125, a consistent reduction of approximately 72 to 76 percent. This difference illustrates the power of heuristic guidance: a well-designed heuristic enables the algorithm to focus on promising regions of the search space rather than exploring blindly (Russell & Norvig, 2020, p. 116).

Seed 50 produces the most instructive result. DFS expanded 7529 nodes under that seed, identical to BFS, yet still returned a suboptimal path of 408 steps. This means DFS explored nearly the entire reachable grid before finding a solution, offering no computational advantage over BFS whatsoever while still failing to guarantee optimality. It is the clearest demonstration in this experiment that a lower or equal node count in DFS carries no meaningful efficiency benefit.

A* nodes visited showed moderate variation across seeds: 1800 (seed 21), 1832 (seed 4), and 2125 (seed 50). Seed 50 required the most expansions, consistent with its noise distribution blocking several promising corridors and forcing A* to explore more widely before converging on the optimal path.

4.4 Explanation of Path Suboptimality in DFS

DFS suboptimality arises from two interacting factors: its depth-first commitment strategy and the presence of the U-trap. The extent of the suboptimality, however, varies considerably with the seed.

From the start at (20, 50), the start node is 30 columns to the left of the back wall, DFS is not immediately blocked by the trap. However, moving right leads toward the open interior of the U-trap. Given the neighbour expansion order (Up, Down, Left, Right), DFS will first explore upward and then eventually rightward. When it enters the trap interior (x between 31 and 49, y between 31 and 69), it continues pushing eastward until it hits the back wall at $x = 50$. At that point it backtracks and systematically explores the entire interior before finally escaping.

Once outside the trap, DFS commits to the first outward path it finds. The length of this path depends heavily on which corridors noise has opened or closed. Under seed 4, noise created a configuration that forced DFS into an extremely long detour of 1584 steps, fifteen times the optimal. Under seed 21 the detour was 746 steps, and under seed 50 DFS happened to encounter the goal relatively early in its post-

trap exploration at 408 steps, though only after exhausting 7,529 nodes in the process.

This behaviour illustrates a key point: a shorter DFS path does not mean DFS was more efficient under that seed. Under seed 50, DFS was actually at its worst in terms of nodes expanded, spending the most computational effort of any DFS trial before finding a solution. The apparently shorter path was a consequence of where the goal happened to fall relative to DFS's exhaustive traversal, not any improvement in the algorithm's strategy. This is precisely why DFS is unsuitable for shortest-path problems regardless of the noise configuration (Russell & Norvig, 2020, p. 98).

4.5 Comparison of Nodes Visited

The disparity in nodes visited between BFS and A* highlights the impact of heuristic guidance. BFS blindly expands all nodes at depth d before moving to depth $d + 1$. Consequently, it explores a huge number of nodes in directions that do not lead toward the goal. In this problem, the optimal path requires going either above or below the trap; BFS explores both options equally, wasting effort on the side that will eventually be abandoned.

A*, on the other hand, prioritises nodes that appear to be closer to the goal (*lower h*). The Manhattan distance heuristic strongly favours nodes with smaller horizontal distance to the goal. Thus, A* quickly focuses on the region near the optimal path and expands far fewer nodes overall.

DFS's node count is intermediate but misleading: it visited many nodes inside the trap before backtracking. Those nodes are "wasted" in the sense that they do not contribute to the final (suboptimal) path; they are merely part of the search history.

4.6 Impact of the Heuristic on Computational Performance

The impact of the heuristic on A*'s performance is clearly demonstrated by the results. The Manhattan distance is a consistent heuristic, as it satisfies the triangle inequality. This property ensures that A* is optimally efficient (Russell & Norvig, 2020, p. 106). The reduction in nodes expanded, from 7315 with Breadth-First Search (BFS) to 1800 with A*, represents a 75% savings. This reduction directly leads to faster execution times, with A* taking 3 ms compared to 6 ms for BFS. Although the absolute execution times are small, the relative difference is significant and would become even more pronounced on larger grids.

4.7 Influence of the Seed Value

The experimental data from seeds 4, 21, and 50 reveal several important patterns.

Both BFS and A* search algorithms showed stable and optimal path costs across all three seeds, producing either 104 or 106 steps depending on the seed. The slight variation between seeds can be attributed to noise that occasionally obstructed the single shortest route, necessitating a one-step detour. Ultimately, the optimal path length is determined by the fixed U-trap geometry rather than the noise configuration.

The cost of the Depth-First Search (DFS) path varied significantly across different seeds: it took 1584 steps for seed 4, 746 steps for seed 21, and 408 steps for seed 50. This variation is the most substantial across the entire experiment. Seed 4 represented the worst-case scenario for DFS, resulting in a path that was fifteen times longer than the optimal route. Conversely, seed 50 was the best-case scenario for DFS in terms of path cost; however, this advantage came with the drawback of exploring 7529 nodes, the same number as the Breadth-First Search (BFS), and taking 18 ms, by far the longest execution time recorded. This outcome is particularly telling: for seed 50, DFS effectively resembled a near-exhaustive search of the reachable grid before finally finding the goal. The path was shorter, not because DFS

was efficient, but because the first solution it discovered late in its search happened to be less circuitous under that specific noise configuration.

The number of nodes visited by the A* algorithm showed moderate variation: 1800 for seed 21, 1832 for seed 4, and 2125 for seed 50. Seed 50 required the most A* expansions, indicating that its noise distribution obstructed several promising pathways close to the optimal route. As a result, A* had to explore more widely before reaching a solution. Nevertheless, A* still expanded significantly fewer nodes than BFS for every seed tested, and its execution time never exceeded 5 ms. This demonstrates that, even in a relatively unfavourable noise configuration, A* with an admissible heuristic is both optimal and computationally superior to BFS (Russell & Norvig, 2020, p. 106).

The result from the seed 50 analysis also highlights an important edge case: when both Depth-First Search (DFS) and Breadth-First Search (BFS) visit the same number of nodes, it suggests that DFS has nearly explored all reachable states before discovering the goal. In these situations, DFS provides no computational advantage over BFS and still does not guarantee finding the optimal path.

5. Conclusion

5.1 Summary of Findings

This investigation compared Depth-First Search, Breadth-First Search, and A* Search in a 100×100 grid containing a U-shaped obstacle and 15% random noise, tested across three seed values (4, 21, and 50). The results confirm the theoretical properties discussed in Russell & Norvig (2020, Chapter 3).

- DFS was not optimal under any seed. Path costs ranged from 408 to 1584 steps between four and fifteen times longer than the optimal solution, depending on the seed. Node counts ranged from 2892 to 7529, and under seed 50, DFS expanded as many nodes as BFS while still failing to return the shortest path. This demonstrates that a high node count in DFS does not correlate with solution quality; it simply reflects how deeply and wastefully the algorithm explored before finding any path.
- BFS was optimal under all three seeds, returning paths of 104 steps for seed 21 and 106 steps for seeds 4 and 50. Node counts ranged from 7315 to 7529, reflecting its exhaustive layer-by-layer expansion. While BFS guarantees optimality for uniform-cost problems, it does so at the expense of exploring the largest number of nodes of any algorithm tested.
- A* was both optimal and the most computationally efficient algorithm in every trial. It found the same optimal paths as BFS: 104 steps for seed 21 and 106 steps for seeds 4 and 50, while expanding only 1800 to 2125 nodes, a reduction of approximately 72 to 76 percent compared to BFS. Execution time never exceeded 5 ms across any seed. These results confirm that informed search with an admissible heuristic dramatically outperforms uninformed search when domain-specific knowledge is available, and that this advantage is consistent and reproducible across different noise configurations.

5.2 Limitations and Future Work

Several limitations of the current study should be acknowledged:

- The grid size (100×100) is relatively small by modern standards. Future work could scale up to larger grids (e.g., 1000×1000) to observe the algorithms' behaviour under more demanding conditions.
- Only one seed (21) was used; a comprehensive analysis would involve multiple seeds and statistical aggregation of results to account for variability in obstacle placement.
- The heuristic used was the simple Manhattan distance. More sophisticated heuristics, such as those derived from pattern databases or landmarks (Russell & Norvig, 2020, p. 118), could be investigated to see whether further efficiency gains are possible.
- The environment is static and deterministic; extensions to dynamic or stochastic environments would require more advanced techniques (e.g., adaptive A or D Lite).

Despite these limitations, the current work provides a solid foundation for understanding search algorithms and their application to real-world problems. It also serves as a stepping stone to more advanced topics in the COS314 curriculum, including game playing, metaheuristics, and machine learning.

6. References

1. Russell, S.J. & Norvig, P. (2020) Artificial Intelligence: A Modern Approach. 4th edn. Harlow: Pearson. (Global Edition)
2. University of Pretoria (2026) COS314 Study Guide: Artificial Intelligence. Department of Computer Science, University of Pretoria.
3. Koza, J.R. (1992) Genetic Programming: On the Programming of Computers by Means of Natural Selection. Cambridge, MA: MIT Press. (Mentioned in relation to evolutionary algorithms, though not directly used, it provides background.)
4. McCarthy, J. (1958) 'Programs with Common Sense', Proceedings of the Symposium on Mechanisation of Thought Processes, National Physical Laboratory, Teddington, UK, pp. 77–84. (Historical context.)
5. Nilsson, N.J. (1971) Problem-Solving Methods in Artificial Intelligence. New York: McGraw-Hill.